ECE 251: Computer Architecture

Week 13: Memory Hierarchies (Part 1)

Prof Rob Marano

Spring 2026

Agenda

1. 1. Introduction to Memory Hierarchy
2. 2. Memory Technologies
3. 3. The Basics of Caches
4. 4. SystemVerilog Hardware Implementations

1. The Journey: From Processor to Memory

- **Where we've been:** We built the logic gates, the ALU, the single-cycle datapath, and finally the pipelined processor with hazard detection.
- **The Final Frontier:** A fast CPU is useless if it has no data. We must now design the Memory System.
- **The Historical Problem:** Early computers had simple, flat memory. Over time, processors became exponentially faster, but memory speeds lagged. CPUs were starving for data!
- **The Solution:** Architects invented the **Memory Hierarchy** (Caches, Main Memory, Virtual Memory) to bridge this chasm.

1. The Processor/Memory Paradox

- Computer architects face a fundamental physical paradox:
- **Processors are incredibly fast** (Clock cycles in nanoseconds or picoseconds).
- **Memory is relatively slow** (Access times in tens of nanoseconds or milliseconds).
- If the CPU has to wait for Main Memory on every instruction fetch and data load, the massive speed gains of pipelining ($CPI \approx 1.0$) are entirely lost!

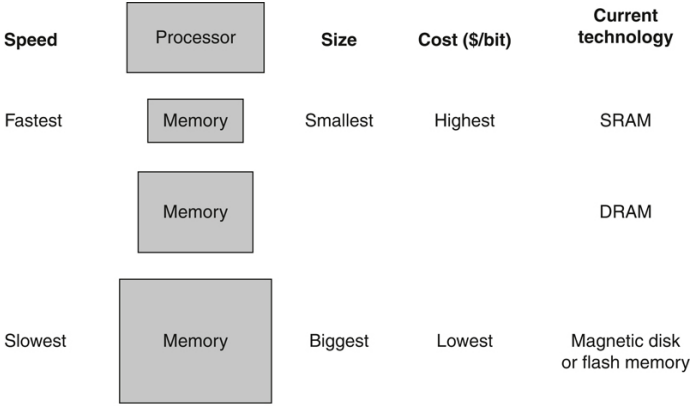
1. The Principle of Locality

- To keep the processor fed with data without stalling, we rely on the **Principle of Locality**:

- **Temporal Locality (Locality in Time):**
  - If an item is referenced, it will tend to be referenced again soon.
  - *Example:* Loops iterating over the same variables.

- **Spatial Locality (Locality in Space):**
  - If an item is referenced, items whose addresses are close by will tend to be referenced soon.
  - *Example:* Sequentially accessing elements in an array.

1. The Memory Hierarchy Pyramid



Copyright © 2021 Elsevier Inc. All rights reserved

- We exploit locality by implementing multiple tiers of memory.
- **Closer to CPU**: Faster, smaller, more expensive per bit.
- **Further from CPU**: Slower, larger, cheaper per bit.

1. Key Terminology

- **Block (or Line):** The minimum unit of information that can be either present or not present in a two-level hierarchy.
- **Hit:** Data requested by the processor appears in some block in the upper level.
  - **Hit Rate:** The fraction of memory accesses found in the upper level.
  - **Hit Time:** The time required to access data in the upper level.
- **Miss:** Data is not found in the upper level.
  - **Miss Penalty:** The time required to fetch the block from the lower level into the upper level, plus the time to deliver it to the processor.

1. Harris Perspective: The "Desk" Analogy

- *Harris & Harris* provide a highly physical analogy for the memory hierarchy:

- **The CPU**: You, sitting at a desk, actively working on a problem.
- **The Cache**: The physical surface of your desk. Small, but you can access any paper on it instantly (SRAM).
- **Main Memory**: A filing cabinet across the room. It holds much more data, but retrieving a folder requires you to stand up and walk over (DRAM Miss Penalty).
- **Magnetic Disk**: The warehouse down the street. It holds everything, but takes a massive amount of time to fetch.

1. Average Memory Access Time (AMAT)

- How do we mathematically measure the performance of this hierarchy?
- **AMAT Formula:** `Hit Time + (Miss Rate × Miss Penalty)`
- *Example*: If cache hit time is 1 cycle, miss penalty is 100 cycles, and miss rate is 5%:
- `AMAT = 1 + (0.05 × 100) = 1 + 5 = 6 cycles`

2. Memory Technologies: SRAM vs DRAM

- **SRAM (Static RAM):**
  - Used for caches (L1, L2, L3).
  - Fast (0.5ns - 2.5ns access time).
  - Requires no refreshing to keep data (hence "static").
  - Low density and very expensive (6 transistors per bit).
- **DRAM (Dynamic RAM):**
  - Used for Main Memory.
  - Slower (50ns - 70ns access time).
  - Data stored as a charge on a capacitor; must be periodically "refreshed" because charge leaks.
  - High density and cheaper (1 transistor per bit).

2. Deep-Dive: Inside a DRAM Cell

- **1T1C Architecture:** 1 Transistor + 1 Capacitor per bit. Massive density.
- **Destructive Reads:** Reading the capacitor drains its tiny electrical charge. The sense amplifiers must immediately rewrite the data back into the cell.
- **Refresh Cycles:** Capacitors leak! The memory controller must halt the CPU and *refresh* (read and rewrite) every row every $\approx$ 64ms to prevent data corruption.
- **Multiplexing:** Row Address Strobe (RAS) and Column Address Strobe (CAS) send the row and column halves of the address over the exact same pins to save space on the physical chip.

2. Non-Volatile Memory

- **Flash Memory:**
  - Non-volatile semiconductor memory (EEPROM).
  - Survives power-off.
  - Reads are fast, but writes are significantly slower and physically wear out the memory cells over time (requires wear leveling).
- **Magnetic Disk:**
  - Used for massive secondary storage.
  - Relies on moving mechanical parts (platters and read/write heads).
  - Measured in milliseconds (millions of times slower than SRAM).

2. Hamacher Perspective: Write Policies

- When the CPU executes a `sw` (Store Word), data is written to the Cache. But when does it update Main Memory?
- **Write-Through:** Data is written to *both* the cache block and the lower-level memory simultaneously. Perfectly safe, but extremely slow.
- **Write-Back:** Data is written *only* to the cache. The block is marked with a **Dirty Bit**. It is only written to Main Memory if that block is evicted later. Incredibly fast, but vulnerable to power loss.

2. Hamacher Perspective: Memory Interleaving

- To reduce the **Miss Penalty** when fetching a block from slow DRAM, *Hamacher* introduces **Interleaving**.
- Instead of storing sequential blocks on one memory chip, the memory controller distributes consecutive blocks across multiple discrete memory banks.
- The CPU can send fetch requests to all banks simultaneously, receiving the data in parallel rather than sequentially!

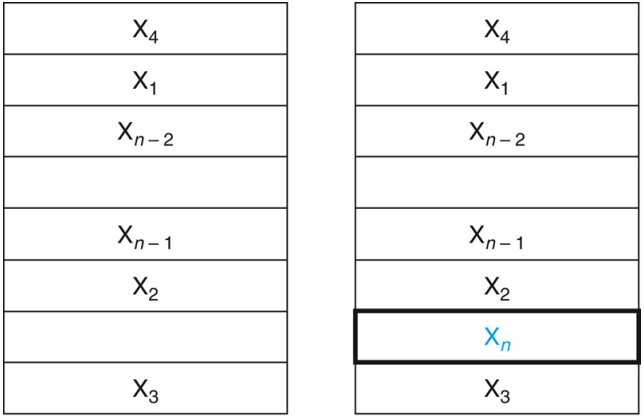
3. Mapping Memory to Cache

- The **Cache** is the level of the memory hierarchy closest to the CPU.
- How do we map memory blocks from the massive Main Memory into the tiny Cache?
- There are three fundamental topologies, trading off hardware complexity against miss rates:
  1. **Direct-Mapped Caches**
  2. **Fully Associative Caches**
  3. **Set-Associative Caches (N-Way)**

3. Direct Mapped Caches

- In a direct-mapped cache, each memory block maps to exactly *one* specific block in the cache.
- **Mapping Formula:** `(Block address) modulo (Number of blocks in the cache)`
- **Address Bit Breakdown:**
  - **Offset:** The lowest bits. Identifies specific byte. Width = $\log_2$(Bytes per Block).
  - **Index:** The middle bits. Identifies exact cache row. Width = $\log_2$(Number of Cache Blocks).
  - **Tag:** Leftover upper bits stored alongside data to verify a match.
- **Pros/Cons:** Extremely fast hit time (only one tag to check). Very cheap. Prone to high *Conflict Misses*.

3. Direct Mapped Cache Architecture



a. Before the reference to X_n b. After the reference to X_n

Copyright © 2021 Elsevier Inc. All rights reserved

3. Fully Associative Caches

- To solve conflict misses, a **Fully Associative** cache allows any memory block to be placed in *any* available cache block.
- **Mapping:** There is no Index field! The physical address only contains the **Tag** and **Offset**.
- **Hardware Cost:** The cache must compare the requested Tag against *every single Tag in the cache simultaneously*. Requires massive, expensive parallel hardware comparators.
- **Pros/Cons:** Lowest possible miss rate. Incredibly expensive and consumes significant power. Restricted to very small memory structures (like TLBs).

3. Set-Associative Caches (N-Way)

- The "Goldilocks" compromise. The cache is divided into "Sets", and each set contains N cache blocks (e.g., 2-way, 4-way, 8-way).
- **Mapping:** A memory block maps to exactly *one* Set (using the Index), but can be placed into *any* of the N blocks within that set.
- **Address Bit Breakdown:**
  - **Offset:** $\log_2$(Bytes per Block).
  - **Index:** $\log_2$(Number of Sets).
  - **Tag:** Leftover bits.
- **Pros/Cons:** Greatly reduces conflict misses compared to Direct-Mapped, without the insane hardware costs of Fully Associative. Uses LRU replacement.

4. Level 1: Basic Synchronous Memory

- How do we build Main Memory in RTL? As a massive array of registers.

```
module main_memory (
    input  logic        clk,
    input  logic        we,       // Write Enable
    input  logic [31:0] addr,
    input  logic [31:0] wd,       // Write Data
    output logic [31:0] rd        // Read Data
);
    logic [31:0] mem [0:1023];    // 4KB Memory Array

    assign rd = mem[addr[31:2]];  // Asynchronous Read (Word aligned)

    always_ff @(posedge clk) begin
        if (we) mem[addr[31:2]] <= wd; // Synchronous Write
    end
```

4. Level 2: Direct-Mapped Cache Arrays

- A Direct-Mapped Cache requires three distinct physical arrays:

```
// Parameters for a small 4KB Direct-Mapped Cache
// Block Size = 1 Word (4 Bytes)
// Number of Blocks = 1024

logic [31:0] cache_data  [0:1023]; // The actual data blocks
logic [19:0] cache_tags  [0:1023]; // The upper 20 bits of the address
logic        cache_valid [0:1023]; // 1-bit valid flags
```

4. Level 3: Combinational Hit/Miss Logic

- The Cache Controller must slice the address and determine a Hit.

```
// Step 1: Slice the 32-bit CPU physical address
assign index = cpu_address[11:2];  // 10 bits for 1024 blocks
assign tag   = cpu_address[31:12]; // Remaining 20 upper bits

// Step 2: Extract the stored Tag and Valid bit using the Index
assign stored_tag = cache_tags[index];
assign is_valid   = cache_valid[index];

// Step 3: Hit Logic (Combinational)
assign cache_hit = is_valid & (stored_tag == tag);

// If cache_hit == 0, trigger the FSM to stall and fetch from DRAM!
```

Questions?